Programming ESP8266 - V2

EE 470 - Introduction to the Internet of Things- Fall 2026

NAME _Antonio Kassis_ DATE: _11/9/2026_

This project must be done individually!

Link to this document:
https://docs.google.com/document/d/1rjYxMpNiV-YkH1WDfW7rW49njwo4tJZoLVA5Ixw6gWk/edit?usp=sharing

Please make sure you submit your assignment individually!

Programming ESP8266 - V2

EE 470 - Introduction to the Internet of Things- Fall 2026

1. Assignment Overview

The purpose of this exercise is to understand various functionalities of ESp8266 and learn more advanced programming topics.

2. Learning Objectives

- Class & Struct in programming
- Connecting to WiFi
- Learning about API
- Extracting JSON format
- Learn Visual Studio & Platform IO

3. Required Materials & Reference Information

- Read the lecture slides carefully.
- Review the samples file on GitHub.
- Visual Studio Code
- ESP8266 Module
- One Switch, One title sensor, one or two ambient sensors (and/or) a proximity sensor.

4. Experiment

~~4.1 Creating a Class (Ignore section 4.1 and go to 4.2)~~

~~In this exercise we would like to create a class in which we can use the following commands to create objects and blink the appropriate LED at a selected rate in milliseconds. In this case, the name of the class must be Blink. The object name must be LED. The user must be able to select the pin number and the blinking rate:~~

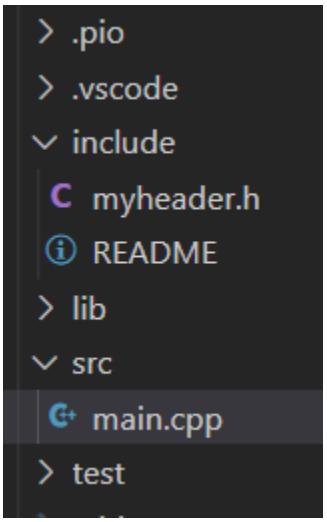
- ~~`Blink LED(pinNumber);`~~
- ~~`LED.blinkRate(250);`~~

~~The Blink class must have the following public and private members:~~

```
4   class Blink {
5   public:
6       Blink(int pin);
7       void blinkRate(int rate);
8   private:
9       int _pin;
10  };
```

Node that you must have TWO files: `ledblink.h` and `ledblink.cpp`. Your `main.cpp` must execute the class. **DO NOT CHANGE the file names or the class name.**

Show a snapshot of your file structure in VSC.



Snapshot of your file structure in VSC - something like this. Node that you must have TWO files: ledblink.h and ledblink.cpp. Your `main.cpp` must execute the class. **DO NOT CHANGE the file names or the class name.**

Show snapshots of your code. You need to have the proper header format for each file to receive a grade.

Snapshot of your codes (`.h` and `.cpp` and the `main.cpp`). THREE FILES

Questions about Class

Answer the following questions.

1- What is the difference between a class and struct?

2- In the main.cpp file what file format with what extension type do you include?

3- What do we call #include?

4- Can you define a function in class?

5- How much RAM is your program using?

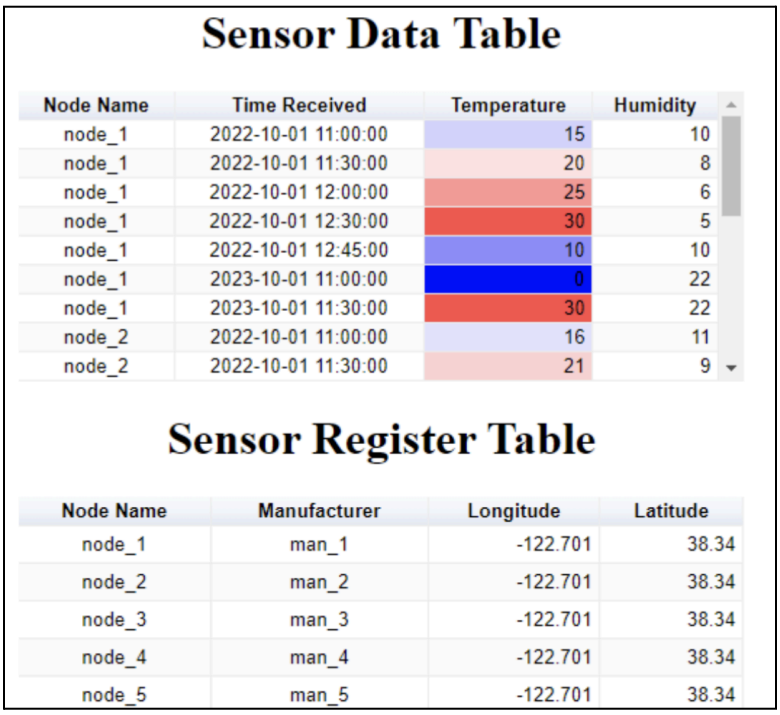
6- How much of FLASH space is your program using?

4.2 Using 8266 to Transmit Data

In this section we would like to design and implement an ESP-based IoT system that collects sensor data, transmits it reliably to a database, and displays real-time measurements through an interactive webpage.

4.2.1 Preparing PHP file

In the previous assignment you created a database for your sensors. You should have two tables in your database, similar to the figure below: **sensor_data and sensor_resister**. Note that the input data must be accepted only if the sensor is **already registered** in your table.



Sensor Data Table

Node Name	Time Received	Temperature	Humidity
node_1	2022-10-01 11:00:00	15	10
node_1	2022-10-01 11:30:00	20	8
node_1	2022-10-01 12:00:00	25	6
node_1	2022-10-01 12:30:00	30	5
node_1	2022-10-01 12:45:00	10	10
node_1	2023-10-01 11:00:00	0	22
node_1	2023-10-01 11:30:00	30	22
node_2	2022-10-01 11:00:00	16	11
node_2	2022-10-01 11:30:00	21	9

Sensor Register Table

Node Name	Manufacturer	Longitude	Latitude
node_1	man_1	-122.701	38.34
node_2	man_2	-122.701	38.34
node_3	man_3	-122.701	38.34
node_4	man_4	-122.701	38.34
node_5	man_5	-122.701	38.34

Figure 1: Example of Sensor Table.

Furthermore, you should be able to plot your data:

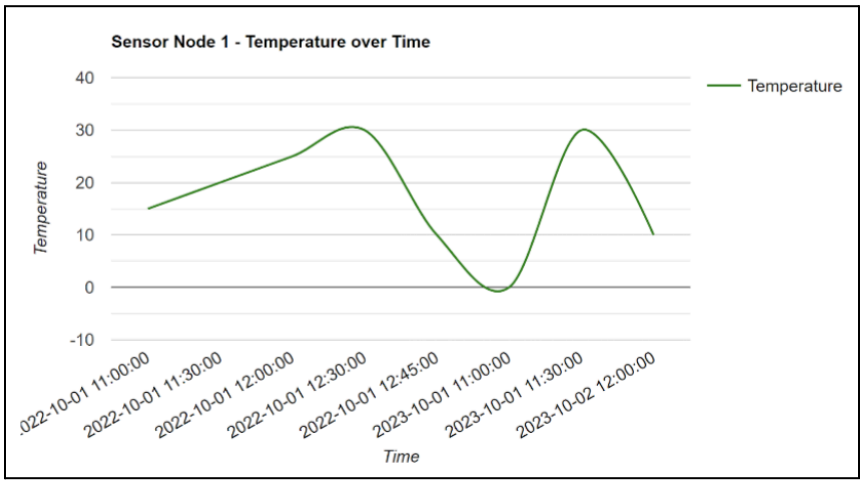


Figure 2: Example of plotted data.

Use the example files on GitHub to ensure your database operates properly.

4.2.2 Design Requirements

In this assignment, your design must meet the following requirements:

1. **Development Environment:** You must use **Visual Studio Code (VS Code) and PlatformIO** to develop your ESP code.
2. **Hardware:** Your microcontroller must be connected to the following:
    - **TWO physical sensors** (temperature, humidity, light, proximity, etc.) - be creative and try to make your design useful! Whatever sensor you choose, remember you need to be able to change the value or the sensor. How do you plan to change the value of the humidity sensor? 😕
    - You must also have two switches: **ONE pushbutton switch**, and **ONE tilt sensor** (be creative! 😊).
3. **Database:** Your database must contain THREE tables:
    - **SensorRegister** and **SensorData:** Their contents are shown in Figure 1.
    - **SensorActivity:** Records the number of valid, unique messages received from each node. The counter must increase **only when a new record is successfully stored**.
4. **Timestamp:** Your microcontroller must generate the **actual date and time** when each measurement is taken. You may use timeapi.io, as explained in the lecture.
    - If the timestamp is missing from the URL, the data **must be rejected**.
    - The database must **reject duplicate timestamps from the same node**.
    - Through the **USB serial interface**, the user must be able to select the time zone so the correct time is transmitted. **Read Appendix A carefully.**
5. **Data Transmission:** Sensor data must be transmitted **ONLY when one of the switches is activated**:

- **Pushbutton pressed:** Data is sent as **Node_1**.
    i. Note: Holding button should not result in sending multiple data points.
    ii. You MUST use a switch! Do not use wires!
- **Tilt sensor activated:** Data is sent as **Node_2**.
- **Each new activation must trigger only ONE transmission.**
- In either case (pressing switch 1 or 2) the transmitted data must indicate your actual sensor value.

6. **Data Storage:** All accepted measurements must be stored in the **SensorData** table. Each record must include **node_number, time, and both sensor values**.
7. **Node Registration:** Only data from nodes registered in **SensorRegister** may be accepted into the database.
8. **RSSI:** Make sure your ESP can display the RSSI value using the USB interface.
9. **Automatic Retransmission:** If a transmitted message is not successfully received and accepted by the server, the ESP must automatically **retry sending the same message within the next 60 seconds**.
    - The ESP must check the **HTTP server response** to determine whether the message was successfully received.
    - The retransmitted message must contain the **original timestamp and sensor values**.
    - Retransmission must continue every **30 seconds** until successful delivery is confirmed.
    - If the server indicates that the message has already been stored, it must be considered successfully delivered to avoid duplicate records.
10. **Website:** Your webpage must display updated sensor information (automatically or through manual refresh), including:
    - **Statistics:**
        i. **Averages, Min, Max -** For example, Average_Temperature and Average_Humidity (or averages of your selected sensors).
    - **Count:** The number of valid, unique measurements received from each node.
    - **Plot:** New measurements must appear on the plot when the webpage is updated (Figures 1 and 2).
    - **Dropdown menu:** Allows users to select a registered node.
    - **Node-specific plot:** Selecting a node must display only that node's measurements.

4.2.3 Code Structure in ESP

PAY ATTENSION: Your ESP code must follow a modular programming approach. Instead of writing all the code inside `loop()`, divide your program into separate functions, each responsible for a specific task:

- Your `loop()` must primarily contain function calls, not lengthy blocks of code.

- Each function must perform a specific task (e.g., reading a sensor, checking a switch, or transmitting data).
- You may define additional functions as needed.
- Your program must ensure that only one transmission occurs per new pushbutton press or tilt activation.
- Functions should be clearly named and organized so the code is easy to read, understand, test, and maintain.

```
void setup() {
  initialize_sensors();
  initialize_wifi();
  initialize_serial();
}

void loop() {
  check_switch();
  check_tilt();
  read_time();
  read_sensor_1();
  read_sensor_2();
  transmit();
  check_error();
}
```

Here is a general example - **Important:** This example illustrates code organization, not the exact execution sequence. Your implementation must ensure that sensor readings and data transmission occur only when an activation is detected.

4.2.3 Demonstration Requirements

Your demonstration will be assessed based on the following.

1. Clearly demonstrate all the requirements in the Design Requirement Section, above.
2. Make sure your sensor data can be changed.
3. Check all your hardware connected to the ESP module and make sure they operate properly.
4. You must be able to explain how your ESP code works! Pay attention to this.
5. Make sure your timestamp is correct when you transmit data and your timezones operate properly.
6. Check and confirm pressing and holding the button will not result in multiple transmissions.
7. Confirm your retransmission operates properly.
8. Be able to show your code. Your code must have proper comments.
9. Show your code using Visual Code.
    a. Your code must have proper comments. Make sure your code has a **header** and in the header include **version, input output** - See appendix B
    b. You must demonstrate your module code, as described above.
10. Make sure you can clearly explain how your code works! You will not receive a grade if you cannot clearly explain how your code operates! Review your code before demonstrating.

Note: In order to demonstrate your project, you must connect your ESP to a network at the University. You cannot connect ESP to SSU-SECURE! Use a program such as https://connectify.me/download/ to create a hotspot. In case you are experiencing issues, you can always connect to my personal hotspot.

You can always use Windows to create a hotspot. MAC users can use this tutorial to create a hotspot.

4.2.4 Circuit Diagram

Submit your circuit diagram - Use EasyEDA (https://easyeda.com/). Your circuit must be very clear! Make sure you review the sample schematic on the lecture slides. If the schematic is not readable you will not receive a grade! There is no partial credit! Use correct library parts.

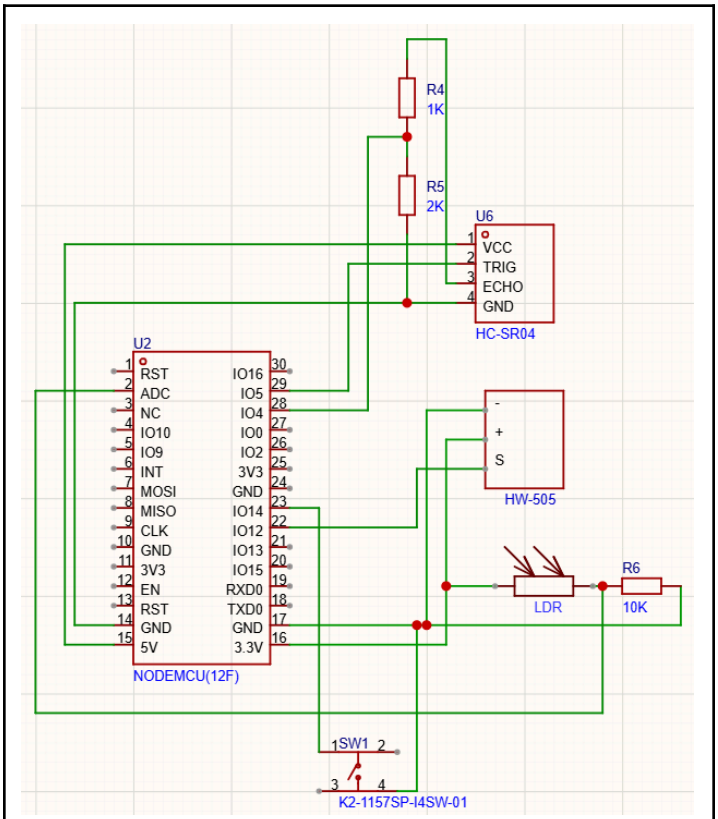


Figure 1: Screenshot of the circuit diagram

The circuit consists of an ESP8266 NodeMCU connected to a photoresistor, HC-SR04 ultrasonic proximity sensor, pushbutton, and tilt switch. The photoresistor is connected as a voltage divider with a 10 kΩ resistor, and the analog voltage is measured through the A0 input. The HC-SR04 uses D1 as the trigger pin and D2 as the echo input. A voltage divider is used on the HC-SR04 echo signal to reduce the 5 V output to a safe voltage for the ESP8266. The pushbutton is connected to D5 and the tilt switch is connected to D6, with both configured using the ESP8266 internal pull-up resistors.

4.2.5 Software Block Diagram

Software Block Diagram: Create a software block diagram illustrating the main functional modules of your system, their interactions, and the flow of data from the ESP to the database and webpage.

Your diagram must include time zone selection, obtaining the current time, reading sensor data, detecting pushbutton/tilt activation, identifying the node, transmitting data, checking the server response, handling errors and automatic retransmission after 60 seconds, validating and storing data in the database, updating node counters, and displaying results on the webpage. Use labeled blocks and arrows to show how these functions interact and how the system makes decisions. Focus on the software operation and logic, rather than hardware connections.

You do not need to include your PHP or database side.

You need to be able to clearly explain this diagram during your project demonstration.

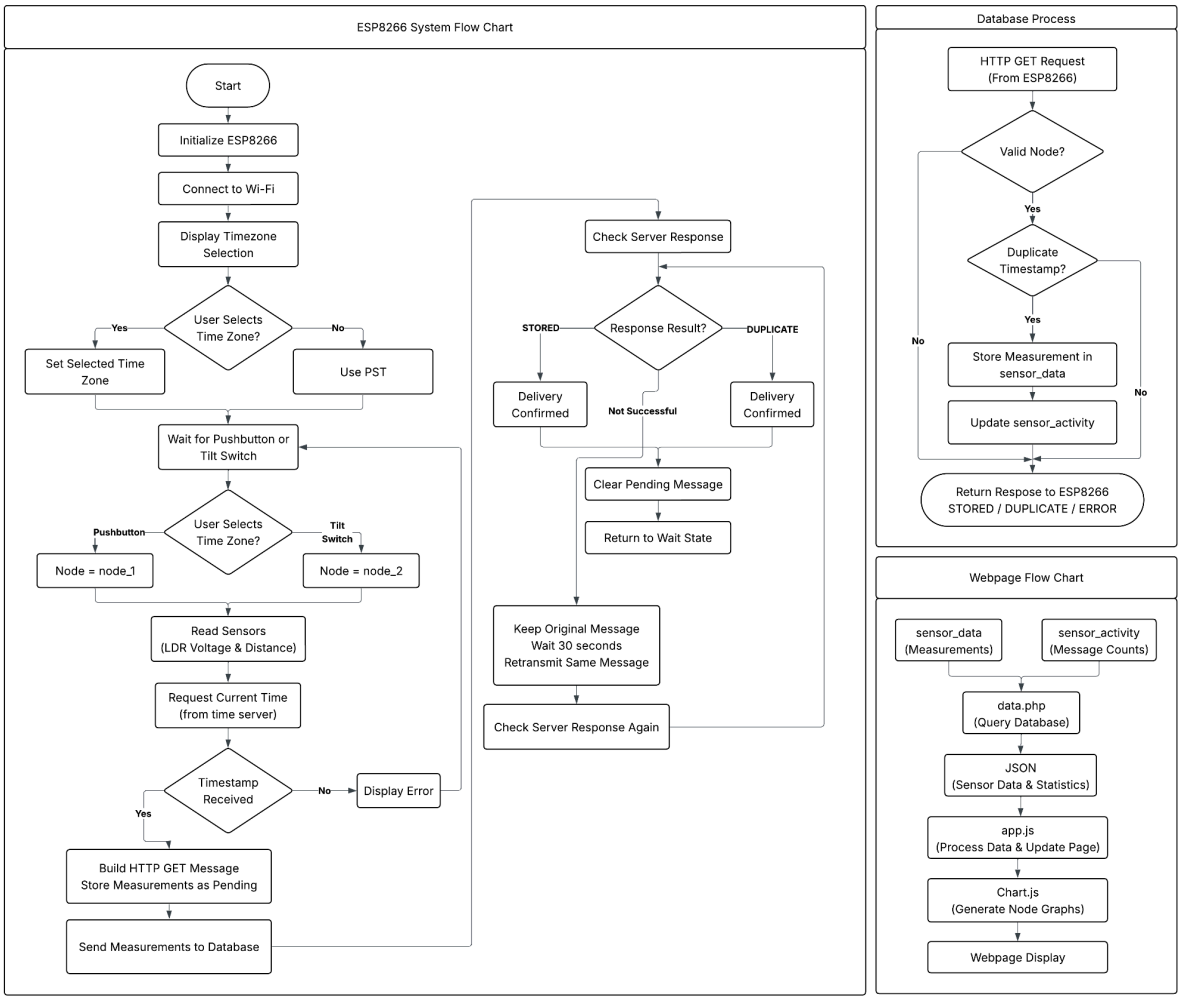


Figure 2: Screenshot of the system flow charts for the ESP, database, and website portions

The software flow chart shows the complete operation of the ESP8266 IoT system from initialization through database storage and webpage display. The ESP8266 first initializes, connects to Wi-Fi, and allows the user to select a time zone, with Pacific Time used as the default if no selection is made. The program then waits for either the pushbutton or tilt switch to be activated. The pushbutton represents node_1, while the tilt switch represents node_2. Once a switch is activated, the ESP8266 reads the photoresistor voltage and HC-SR04 distance, requests the current time from the time server, and verifies that a valid timestamp was received.

After the sensor readings and timestamp are available, the ESP8266 builds an HTTP GET request and stores the measurement as a pending message before sending it to the server. The server checks whether the node is valid and whether the timestamp is a duplicate. If the measurement is new, it is stored in the sensor_data table and the sensor_activity counter is updated. The server then returns a STORED, DUPLICATE, or ERROR response to the ESP8266. A STORED or DUPLICATE response confirms successful delivery and clears the

pending message. If delivery is not successful, the ESP8266 keeps the original measurement, waits 30 seconds, and retransmits the same message until delivery is confirmed.

The webpage portion of the flow chart shows how the stored database information is presented to the user. The sensor_data and sensor_activity tables are queried by data.php, which formats the sensor information and statistics as JSON. The app.js file processes the returned data and Chart.js generates the graphs. The webpage then displays the selected node's measurements, statistics, message count, and sensor graphs.

4.2.6 Questions

1- When you connect the ESP to your WiFi what IP address does it have?
The IP address of my ESP8266 is 10.144.127.214.

2- How much RAM is your program using?
The final program uses 30.3 KB of RAM out of 81.9 KB, which is 37.0%.

3- How much of FLASH space is your program using?
The final program uses 399 KB of Flash out of 1.044 MB, which is 38.2%.

4- What is your microprocessor's MAC address? How do you know?
The MAC address of my ESP8266 is 40:F5:20:2E:2D:26. I found it using WiFi.macAddress() and also confirmed it in the PlatformIO upload output.

5- Are you using the GET or POST method to transmit your data from ESP? What function do you use to transmit the data?
I am using the GET method. The data is included in the URL and transmitted using http.GET().

6- How does your ESP know if the data was successfully accepted or not?
The ESP checks the HTTP response code and the response returned by the server. A response of STORED means the data was successfully saved, while DUPLICATE means the data had already been stored and is also treated as successful. If the response is not successful, the ESP keeps the original message and retries after 30 seconds.

7- When you connect to your WiFi what is the RSSI value as indicated by the ESP? What exactly does RSSI indicate? Show your results. Show the line of code you used to measure RSSI.

The RSSI value was -51 dBm. RSSI stands for Received Signal Strength Indicator and shows the strength of the Wi-Fi signal received by the ESP8266. A value closer to 0 dBm indicates a stronger signal.

The line of code used was:

WiFi.RSSI()

8- If you move away from your router, what happens to the RSSI value?

If I move away from my router (aka my hotspot) then the RSSI value becomes a larger negative value.

9- Based on your answers above, complete the table below. Add units when appropriate.

IP Address of ESP	**10.144.127.214**	RSSI Value	**-51 dB**
MAC Address of ESP	**40:F5:20:2E:2D:26**	FLASH Space Used	**399 KB**

4.2.7 Links to your documents

https://antoniokassis.com/assignments.html
Link to your web page page

https://github.com/Neo4724/IoT_EE_470/tree/main/EE%20470%20-%20Assignment%3A%20[ESP%20Programming]
Place all your codes on your GITHUB page and add the link here.

5. Project Improvements

What would be one or more cool features that can be added to this system in the future to improve its performance? For example, what can be done to improve retransmission?

One improvement would be to allow the ESP8266 to store multiple failed measurements instead of only keeping one pending message. If the Wi-Fi connection is lost for a longer period of time, the ESP could save several measurements in a queue and transmit them once the network connection is restored.

Another improvement would be to make the retransmission system more efficient. Instead of retrying every 30 seconds continuously, the ESP could increase the delay after repeated failures. The system could also send an alert if the sensor remains offline for a long period of time. These changes would make the system more reliable and reduce unnecessary network traffic.

6. Conclusions

In two-three paragraphs in your own words explain what you learned in this activity. What was interesting?

In this assignment, I learned how to use the ESP8266 as part of a complete IoT system. I connected a photoresistor, HC-SR04 proximity sensor, pushbutton, and tilt switch to the ESP8266 and programmed the board using PlatformIO. I also connected the ESP8266 to Wi-Fi, displayed its IP address, MAC address, and RSSI value, and used a time API to get the correct timestamp based on the selected time zone.

I also learned how to send sensor data from the ESP8266 to a PHP webpage and store the information in a database. The system checks that the node is registered, prevents duplicate timestamps, keeps track of valid messages, and displays the data on a webpage with graphs and statistics. Another important part of the assignment was implementing retransmission so that if a message is not successfully received, the ESP keeps the original data and tries again after 30 seconds.

The most interesting part of the project was seeing all of the different parts work together. A physical switch can trigger the ESP8266 to read real sensor values, get the current time, transmit the data over Wi-Fi, store it in the database, and then display the result on the website. This helped me understand how sensors, embedded programming, networking, databases, and web applications are connected in an IoT system.

7. Quick Survey

Answer the following questions, please:

Survey question	Response
On a scale of 1-10 how did you like this exercise? (10 is the best, 1 is the worst)	7
On a scale of 1-10 how much did you learn as a result of completing this exercise? (10 = plenty; 1=very little)	8
How many hours did you spend completing this exercise?	15

🎉 **Congratulations!** You have now designed and built a fully functional, end-to-end IoT system - all by yourself! From sensors and embedded programming to data transmission, databases, and web visualization, you made it all work together. **Amazing work!** 🚀👏

NOTE: All projects must be shown in class! If you cannot demonstrate your project, you do not receive a grade!

Appendix A

Develop a user interface that allows users to select their time zone using an ESP device connected via USB. This interface should provide a list of time zones and retrieve the current time for the selected zone using an external API.

The ESP device must display the following time zone options when connected to a USB port:

```
1) Eastern Time Zone (ET) - New York City, NY
2) Central Time Zone (CT) -Chicago, IL
3) Mountain Time Zone (MT) - Denver, CO
4) Pacific Time Zone (PT) - Los Angeles, CA
5) Alaska Time Zone (AKT) - Anchorage, AK
6) Hawaii-Aleutian Time Zone (HAT) - Honolulu, HI
7) Atlantic Time Zone (AT) - San Juan, Puerto Rico (US territory,
   not part of the contiguous US).
```

The user should be prompted with a message like:

```
—> Select Your Time Zone (The default time is PT):
```

For example: The user will enter the number corresponding to their desired time zone (e.g., "1" for Eastern Time, "4" for Pacific Time). If no input is provided, the system should default to Pacific Time (PT).

Alternatively, If you prefer. you can add a switch and if you press it 1 time, it would correspond to Selection 1, if you press it 2 time2, it would correspond to Selection 2, and so on. You can have three LEDs displaying the choices in binary.

Note that when using the API:
https://timeapi.io/api/Time/current/zone?timeZone=America/$TIMEZONE

The variable $TIMEZONE must correspond to whatever value the user enters:

1. Eastern Time Zone (ET) → America/New_York
2. Central Time Zone (CT) → America/Chicago
3. Mountain Time Zone (MT) → America/Denver
4. Pacific Time Zone (PT) → America/Los_Angeles
5. Alaska Time Zone (AKT) → America/Anchorage
6. Hawaii-Aleutian Time Zone (HAT) → Pacific/Honolulu
7. Atlantic Time Zone (AT) → America/Puerto_Rico

Appendix B

It is important that your code has the proper header.

```
/*
 * ---------------------------------------------
 * Project/Program Name : <Project Name or ESP Application>
 * File Name            : <filename.c>
 * Author               : <Your Name>
 * Date                 : <DD/MM/YYYY>
 * Version              : <Version Number>
 *
 * Purpose:
 *    <A brief description of the purpose of this file or program>
 *
 * Inputs:
 *    <List of input parameters or signals the program handles, or "None">
 *
 * Outputs:
 *    <List of output signals or results, or "None">
 *
 * Example Application:
 *    <Give an example of how this program or function is used in an application>
 *
 * Dependencies:
 *    <List any external libraries or hardware dependencies>
 *
 * Usage Notes:
 *    <Additional notes about the application or special instructions>
 *
 * ------------------------------------------------------------------------
 */
```